



PACCIOLI

“Desarrolla tu potencial, transforma tu mundo”

INSTITUTO TECNOLÓGICO “PACCIOLI” CARRERA DE SISTEMAS INFORMÁTICOS

MANUAL TECNICO DE PLATAFORMA DE APRENDIZAJE ADAPTATIVO CON IA

Postulante:

Ruben Herbas Tudela

Tutor:

Lic. Kevin Camacho Rodriguez

Punata - Cochabamba - Bolivia

2026

INDICE

1. INTRODUCCIÓN.....	1
1.2. Tecnologías Utilizadas	1
1.3. Arquitectura del sistema y organización de módulos.....	3
1.4. Estructura de directorios y organización del proyecto.....	4
1.5. Configuración de variables de entorno.....	5
1.6. Modelo de base de datos relacional y seguridad	6
1.7. Flujos funcionales lógicos y de integración.....	7
1.8. Comandos de ejecución para desarrollo y producción	8
1.9. Observaciones técnicas y notas de mantenimiento.....	9

MANUAL TECNICO DE PLATAFORMA DE APRENDIZAJE ADAPTATIVO CON IA

1. INTRODUCCIÓN

La Plataforma Educativa Adaptativa con IA representa un avance significativo en los entornos virtuales de aprendizaje (LMS), concebida bajo paradigmas modernos de desarrollo web y computación en la nube. Su arquitectura está optimizada para ofrecer respuestas en tiempo real, adaptabilidad pedagógica y una experiencia de usuario fluida y altamente receptiva. El sistema está estructurado para atender de manera diferenciada a los actores clave del ecosistema educativo: por un lado, los estudiantes disponen de un entorno inmersivo para la visualización de contenidos modulares, realización de evaluaciones dinámicas, seguimiento de calificaciones y recepción de retroalimentación automatizada; por otro lado, los profesores cuentan con herramientas avanzadas para la creación de planes de estudio, administración de matrículas, diseño de cuestionarios evaluativos y analítica predictiva del rendimiento académico.

Desde la perspectiva de la ingeniería de software, el sistema adopta una arquitectura desacoplada donde el cliente web ejecuta la lógica de presentación y el enrutamiento de vistas mediante tecnologías cliente de última generación, mientras que la persistencia, la seguridad perimetral y los servicios de autenticación están delegados a un backend como servicio (BaaS) robusto y escalable. Adicionalmente, se integra un servicio proxy intermedio en Node.js para gestionar de forma segura las peticiones hacia modelos de inteligencia artificial generativa, garantizando la privacidad de las claves de API y permitiendo una capa de fallback o respaldo local en escenarios de interrupción de red o fallos en el proveedor externo.

1.2. Tecnologías Utilizadas

Inventario detallado de tecnologías utilizadas

La selección tecnológica responde a criterios estrictos de rendimiento, mantenibilidad, soporte de tipado estático y compatibilidad con ecosistemas de nube modernos:

- **React 19:** Biblioteca principal para la construcción de interfaces de usuario basadas en componentes declarativos y reactivos, aprovechando las últimas optimizaciones de concurrencia y renderizado del DOM virtual.
- **TypeScript:** Superset tipado de JavaScript que aporta robustez al código mediante la definición estricta de interfaces, tipos de datos y estructuras para perfiles, cursos, evaluaciones e inscripciones, reduciendo drásticamente los errores en tiempo de ejecución.
- **ite:** Entorno de desarrollo ultrarrápido basado en ESM nativo y empaquetado con Rollup, que acelera el arranque del servidor de desarrollo y optimiza los tiempos de compilación para producción.
- **React Router:** Librería encargada de gestionar el enrutamiento dinámico del lado del cliente, permitiendo la protección de rutas privadas según el rol del usuario (estudiante o profesor).
- **Framer Motion:** Biblioteca de animaciones declarativas para React, utilizada para transiciones fluidas entre pantallas, modales y elementos interactivos del dashboard.
- **Estilo visual (Tailwind-like CSS):** Sistema de clases utilitarias CSS estructurado para mantener un diseño consistente, limpio y adaptativo (responsive design) en diversas resoluciones de pantalla.

- **Supabase:** Plataforma backend open-source que actúa como motor de base de datos relacional (PostgreSQL), proveedor de autenticación con JSON Web Tokens (JWT) y gestor de políticas de seguridad a nivel de filas (RLS).
- **Inteligencia Artificial (Google Gemini):** Motor de IA generativa integrado mediante llamadas HTTP asíncronas para la creación dinámica de preguntas de opción múltiple y evaluación adaptativa del dominio del estudiante.
- **Dependencias y librerías auxiliares:** `@supabase/supabase-js` para la comunicación con la base de datos, `three` y `@react-three/fiber` para renderizado 3D opcional en componentes visuales, y `lucide-react` para la iconografía del sistema.

1.3. Arquitectura del sistema y organización de módulos

El código fuente del proyecto está organizado de manera modular para garantizar la separación de incumbencias (Separation of Concerns). A continuación se detallan los módulos principales y su rol arquitectónico:

- `App.tsx`: Punto de entrada principal de la aplicación. Configura el árbol de componentes raíz, inicializa los contextos globales y define el enrutamiento condicional que redirige a los usuarios hacia los layouts correspondientes según su rol verificado (panel de estudiante o panel de profesor).
- `AuthContext.tsx`: Componente de contexto global de React que centraliza el estado de autenticación. Gestiona los eventos de inicio de sesión, registro de nuevos usuarios, persistencia de la sesión activa en el almacenamiento local y la recuperación del perfil asociado en la base de datos.
- `supabaseClient.ts`: Módulo de inicialización que instancia el cliente oficial de Supabase utilizando las variables de entorno públicas, exponiendo una interfaz

única para todas las consultas y mutaciones de datos en la base de datos relacional.

- o `geminiService.ts`: Servicio encargado de interactuar con el proxy de IA. Construye los prompts estructurados para la generación de quizzes y procesa las respuestas del modelo, implementando un mecanismo de respaldo local (fallback) que devuelve preguntas predefinidas si el servicio de IA no se encuentra disponible.
- o `register.js`: Servidor independiente desarrollado en Node.js que opera como middleware o proxy local. Expone endpoints específicos para operaciones de registro administrativo,

consultas avanzadas (lookup) y la comunicación segura con la API de Google Gemini utilizando credenciales privilegiadas.

1.4. Estructura de directorios y organización del proyecto

La estructura de archivos del proyecto sigue una convención estándar orientada a proyectos escalables en React y TypeScript:

```
/
├── server/
│   └── register.js           # Servidor Node.js para proxy de IA y
operaciones de registr
├── src/
│   ├── components/         # Componentes reutilizables de UI
(formularios, tarjetas,
│   ├── contexts/           # Proveedores de contexto global
(AuthContext.tsx)
│   ├── lib/                # Configuración de clientes externos
(supabaseClient.ts, esq
│   ├── pages/              # Vistas de la aplicación (login, dashboard,
cursos, quizzes
│   └── services/           # Lógica de negocio, integración de IA y
cálculo de dominio
├── App.tsx                 # Componente principal de enrutamiento y
estructura
├── main.tsx                # Punto de montaje de la aplicación React
├── supabase.sql            # Script SQL con la definición completa del
esquema relacion
├── package.json            # Manifiesto de dependencias y scripts de
ejecución
└── .env.local              # Archivo de configuración de variables de
entorno locales
```

1.5. Configuración de variables de entorno

Para el correcto funcionamiento de la plataforma en entornos de desarrollo y producción, es obligatorio configurar las siguientes variables de entorno en un archivo `.env.local` ubicado en la raíz del proyecto:

Variable de Entorno	Tipo	Descripción detallada
<code>VITE_SUPABASE_URL</code>	Pública	URL del proyecto asignada por Supabase para la conexión segura mediante HTTPS y WebSockets.
<code>VITE_SUPABASE_ANON_KEY</code>	Pública	Clave anónima pública (anon key) utilizada por el cliente frontend para interactuar con la base de datos
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Privada	Llave con privilegios de rol de servicio (service_role), estrictamente destinada para uso en el servidor backend (<code>register.js</code>) para operaciones administrativas.

<code>GEMINI_API_KEY / VITE_GEMINI_API_KEY</code>	Privada/ Opcional	Clave de API de Google Gemini para habilitar la generación avanzada de cuestionarios mediante IA.
<code>REGISTER_SERVER_PORT</code>	Configuración	Puerto en el que se ejecuta el servidor proxy de registro y IA (configurado por defecto en el puerto 8787).
<code>VITE_AI_PROXY_URL</code>	Configuración	URL completa del endpoint del proxy de IA (por defecto <code>http://localhost:8787/generate-quiz</code>).

1.6. Modelo de base de datos relacional y seguridad

El esquema de base de datos está diseñado en PostgreSQL y se inicializa mediante el script

- `supabase.sql` . Consta de las siguientes tablas fundamentales:

- `profiles`: Almacena la información extendida de cada usuario registrado (nombre, correo, rol institucional como 'student' o 'teacher' y fecha de creación).
- `courses`: Contiene la información de los cursos académicos creados por los docentes (título, descripción, identificador del profesor titular).
- `course_topics`: Estructura temática en la que se divide cada curso, permitiendo un desglose granular del contenido pedagógico.

- `quizzes` y `quiz_questions`: Agrupan las evaluaciones y sus respectivas preguntas de opción múltiple vinculadas a un curso o tema específico.
- `quiz_attempts` y `quiz_answers`: Registran el historial de intentos realizados por los estudiantes y el detalle de cada respuesta emitida para su posterior análisis.
- `course_students`: Tabla intermedia que gestiona la relación de muchos a muchos correspondiente a las inscripciones de alumnos en los cursos.
- `student_mastery`: Métrica adaptativa que cuantifica el nivel de dominio conceptual alcanzado por un estudiante en cada tema del curso.
- `grades`: Almacena las calificaciones definitivas obtenidas por los estudiantes en las evaluaciones del sistema.

Seguridad y Row Level Security (RLS): Todas las tablas del esquema implementan políticas estrictas de RLS en Supabase, asegurando que los estudiantes solo puedan acceder a sus propias calificaciones, intentos y cursos inscritos, mientras que los profesores poseen privilegios de escritura y gestión sobre sus cursos y contenidos asignados.

1.7. Flujos funcionales lógicos y de integración

El ciclo de vida de una solicitud en la plataforma sigue patrones bien definidos:

- **Autenticación y Sesión:** Al iniciar sesión, Supabase valida las credenciales y emite un token JWT. `AuthContext` intercepta este estado, consulta la tabla `profiles` y

determina el rol del usuario, actualizando el árbol de componentes para renderizar el panel correspondiente.

- **Generación de Cuestionarios por IA:** Cuando un profesor solicita la creación de un quiz adaptativo, el frontend invoca al servicio `geminiService.ts`, el cual envía la petición al servidor proxy local (`register.js`). Este servidor se conecta de forma segura con la API de Google Gemini, procesa la respuesta en formato estructurado y la devuelve al cliente. Si la API no responde, se activa un banco de preguntas estáticas de respaldo.

1.8. Comandos de ejecución para desarrollo y producción

El proyecto incluye scripts preconfigurados en el archivo `package.json` para gestionar el ciclo de vida del desarrollo:

```
# Instalar todas las
dependencias del
proyecto npm install

# Iniciar el entorno de
desarrollo frontend (Vite)
npm run dev

# Compilar la aplicación para producción (genera archivos
estáticos optimizados) npm run build

# Ejecutar el servidor auxiliar de registro y proxy de IA
npm run register-server
```

1.9. Observaciones técnicas y notas de mantenimiento

Punto de atención en AuthContext.tsx: Durante auditorías de código recientes, se detectó una ligera inconsistencia en algunas consultas del contexto de autenticación donde se alterna el uso de las propiedades `id` y `user_id` al mapear perfiles de usuario. Se recomienda encarecidamente unificar estas referencias a lo largo de los servicios para prevenir errores intermitentes en la carga de perfiles y sesiones.